NAME: M. UMAIR **ROLL NO: 048**

CLASS: BSAI **SECTION:3A**

LAB TASK: 2

TASK:1

PEAS

1. **Delivery Bot**

Performance:

The robot will deliver parcels to the correct door on time. It won't collide or hurt anyone. It will be energy efficient. The parcel will arrive in good condition.

Environment:

Footpaths, corridors, and entrances to buildings. Other people, cyclists, and other robots. Crossings, ramps, doors, and lifts. Bad weather like rain and sunny.

Actuators:

Motors in wheels for movement and turning. Braking. A lockable compartment for the parcel. A speaker/horn, and screen.

Sensors:

Camera. Lidar and ultrasound for distance. GPS, wheel encoders, and IMU. Battery.

Trade-off:

If the robot travels fast it will deliver on time, however it will have a larger braking distance, use more battery, and therefore increase risk of hurting someone. The robot should slow down in busy places, causing a slight delay. However, safety is prioritised.

Assumptions:

The robot will operate inside a mapped area such as a campus or neighborhood. Mainly during the day and in light/moderate weather. Wi-Fi/mobile data connection and if the robot gets stuck a human can be contacted. The robot can only carry one parcel and the recipient will unlock the compartment using a code.

2 **LLM Support Agent**

Performance:

The response given will be accurate and comply with university policies. The student's issue will be resolved in a small number of messages. Fast response time. If it is unsure it will pass it to a human. Student data will remain private.

Environment:

Students messaging via the university portal/app. University documents (handbook, fees/exams, timetables). University systems (timetables, tickets). Staff and helpdesk.

Actuators:

Respond to messages. Create a support ticket. Send emails/notifications to the correct office. Pass to human.

Sensors:

Message input from student. Chat history. Login ID from portal. Search results from documents. Tool responses.

Trade-off:

If the agent always responds automatically students will receive fast responses. However some responses may be incorrect. If it passes to humans every time it is unsure it will be safe. However staff will become overwhelmed and there will be delays. A sensible option would be to respond to common queries and pass to the human if the query relates to grades, fees, or personal issues.

Assumptions:

The student is logged in so the agent knows who they are. The tools available are: (a) search the official handbook and faqs. (b) View read only timetables and course. (c) View read only information relating to the logged in student (courses they're registered on, fee status). (d) Create support ticket. (e) Send email to department. (f) Pass to human. The agent does not have the ability to change grades, registrations, or payments. It also cannot view other students' data. Documents searched will be up to date.

Task 2:

Classifying the Environment

System Observability Randomness Reason

Delivery Bot Partially Observable Stochastic The robot can only see what is in front of it using sensors. The wheels can slip and people can move unpredictably.

Support Agent Partially Observable Stochastic The agent cannot see the true situation of the student. Responses from tools and students can also vary.

2.1 Delivery Bot

Partially Observable: The robot can see its battery level, speed and how far away obstacles are. It cannot see hidden items like if someone is walking out from behind a pillar, a locked door or if the floor is wet. The robot will need to guess based off its limited sensors.

Stochastic: Predictable outcome: When the robot receives a command to brake it will slow down and stop in approximately the expected distance on dry ground. Unpredictable outcome: When it receives a command to move forward 5m it could be

affected by slipping wheels, rough surface or someone crossing causing it to end up in a different place or stop earlier.

2.2 Support Agent

Partially Observable: The agent can see what the message says, chat history and login ID. It cannot see the true intention of the student, how stressed/confused they are and if the documents are outdated. If a student says 'I can't pay' it could be a fee issue, bank issue or portal issue. This cannot be seen.

Stochastic: Predictable Outcome: When creating a ticket a ticket number will normally be created. Unpredictable Outcome: The response will vary. The same question asked can be phrased slightly differently. Search results can vary and the student's next response cannot be predicted.

2.3 Real Environment vs Simplified Simulation

Both systems would be partially observable and stochastic in a real environment as explained above. However, in a simplified simulation this can be altered based off assumptions. For example if simulating the delivery bot it may be simulated on a clean grid map. Here the robot will always move one cell and all obstacles will be visible. Therefore it will be fully observable and deterministic. If simulating the support agent it may have a set list of 20 questions with set answers. This will be almost deterministic.

Based on the assumptions made. As I have assumed we are in the real world with real people, sensors and university systems my answers will be as above. However if the simulation doesn't include any noise, hidden items or random users then the classification will change. This may not be applicable to a real world scenario.

Task 3:

Mode Memory Controller

As I didn't have access to the code file I created the climate controller based off the task description. I assumed if the room wasn't occupied the target would be OFF (priority to occupancy). If the room was occupied and the temperature was greater than 26 the target would be COOL. If less than 20 it would be HEAT. Finally if the temperature was between 20-26 the target would be OFF. The new aspect is self.previous_mode this keeps track of the previous target. The method returns the target and a boolean. Only if the target has changed from the previous mode will a command be sent. The previous mode will start as None so the first command will always be sent.

class ClimateAgent:

def __init__(self):

self.previous_mode = None # Not sending anything yet

def decide(self, temperature, occupied):

```
# Priority to occupancy
if not occupied:
target = "OFF"
elif temperature > 26:
target = "COOL"
elif temperature < 20:
target = "HEAT"
else:
target = "OFF"
# Only send if target has changed
send = (target != self.previous_mode)
self.previous_mode = target
return target, send
agent = ClimateAgent()
tests = [
(24, True), # First percept
(24, True), # Same
(28, True), # COOL
(28, True), # Same
(26, True), # Exactly 26
(20, True), # Exactly 20
(18, True), # HEAT
(18, False), # Not occupied
(30, False), # Not occupied and hot
]
print("Prev | Temp | Occ | Target | Command")
for t, occ in tests:
prev = agent.previous_mode
target, send = agent.decide(t, occ)
print(prev, "|", t, "|", occ, "|", target, "|", "SEND" if send else "NO COMMAND")
```

3.2 Table of Results

Each row shows a percept (temperature and occupied).

No. Previous Mode Temp (°C) Occupied Target Command / Test

1 None 24 Yes OFF SEND (First percept)

2 OFF 24 Yes OFF NO COMMAND (Same)

3 OFF 28 Yes COOL SEND (Mode change)

4 COOL 28 Yes COOL NO COMMAND (Same)

5 COOL 26 Yes OFF SEND (Exactly 26)

6 OFF 20 Yes OFF NO COMMAND (Exactly 20)

7 OFF 18 Yes HEAT SEND (Mode change)

8 HEAT 18 No OFF SEND (Not occupied so overrides heating)

9 OFF 30 No OFF NO COMMAND (Not occupied, still hot)

GITHUB LINK:

https://github.com/umair-900/AI-LAB-TASKS
